

UNIVERSITÀ DEGLI STUDI DI UDINE

DIPARTIMENTO DI SCIENZE MATEMATICHE,
INFORMATICHE E FISICHE

IBML - Internet of Things, Big Data, Machine Learning



Internet of Things Progetto Pet Tracker

Autori

Andrea Gioia, 169484

Alessio Nicodemo, 166972

Mattia Nadal, 167372

Francesco Sabot, 167339

169484@spes.uniud.it

166972@spes.uniud.it

167372@spes.uniud.it

167339@spes.uniud.it

Sommario

Il progetto Pet Tracker propone la realizzazione di un sistema per il tracciamento in tempo reale di animali domestici di taglia media o grande, tramite un dispositivo *embedded* indossabile. L'obiettivo del lavoro è sviluppare una soluzione affidabile, scalabile e a basso consumo per il monitoraggio continuo dell'animale.

La soluzione si basa su una piattaforma ESP32-S3 con connettività LTE e modulo GNSS, in grado di acquisire e trasmettere lo stato della batteria del dispositivo, insieme alla posizione geografica e all'attività motoria dell'animale, verso un *backend* remoto. L'architettura del sistema segue il modello *IoT World Forum* (IoTWF) e integra diverse tecniche di ottimizzazione energetica, tra cui l'utilizzo delle modalità *deep sleep* e *hot start*. I dati raccolti vengono elaborati dal *backend*, resi accessibili tramite API REST e visualizzati attraverso un'applicazione mobile dedicata.

Indice

1	Hardware	4
1.1	LilyGo T-SIM7670G-S3	4
1.2	Antenne LTE e GNSS	4
1.3	Batteria e power management	4
1.4	BMA400 e CJMCU-400	4
2	Firmware e logica di funzionamento	4
2.1	Gestione della persistenza e hot start	4
2.2	Protocollo di comunicazione HTTPS	5
2.3	Risparmio energetico	5
2.4	Acquisizione GPS e timeout	6
2.5	Connessione alla rete LTE	6
3	Backend e database	6
3.1	Architettura di hosting e affidabilità del sistema	7
3.2	Modellazione dei dati e API rules	7
3.3	Pulizia e normalizzazione (Server-side Hooks)	8
3.4	Macchina a stati delle attività	8
3.5	Lifecycle del deep sleep e servizi temporali	9
3.5.1	Gestione del risveglio e ripristino dello stato	9
3.5.2	Watchdog e split notturno	10
4	Frontend e applicazione mobile	10
4.1	Architettura a livelli	11
4.2	Autenticazione e persistenza della sessione	11
4.3	Splash screen e caricamento parallelo	11
4.4	Comunicazione in tempo reale	11
4.5	Geofencing e geocoding	12
4.6	Schermate e funzionalità principali	12
4.6.1	Analisi delle attività	12
4.6.2	Mappe in tempo reale	13
4.6.3	Gestione delle zone sicure	14
5	Sistema di notifiche push	14
5.1	Analisi dei vincoli tecnologici	14
5.2	Architettura del FCM Bridge	15
5.3	Logica di invio e trigger	15
5.4	Gestione permessi	16
6	Vincoli di sistema	16
7	Sviluppi futuri e prospettive del progetto	17
7.1	Evoluzione e scalabilità del backend	17
7.2	Evoluzione e costruzione hardware	17
8	Conclusioni	18

1 Hardware

1.1 LilyGo T-SIM7670G-S3

Il dispositivo utilizza una scheda LilyGo T-SIM7670G-S3 [1] basata sul microcontrollore ESP32-S3 dual-core. Quest'ultimo gestisce la logica applicativa e la comunicazione con il modem SIM7670G tramite interfaccia UART.

1.2 Antenne LTE e GNSS

Il sistema utilizza un'antenna flessibile LTE [2] per la connettività dati e un'antenna ceramica attiva [3] per il ricevitore GNSS integrato, garantendo la precisione della localizzazione all'aperto.

1.3 Batteria e power management

Il sistema è alimentato da una batteria ricaricabile agli ioni di litio da 3000 mAh e 30 A. Il monitoraggio della carica avviene tramite un partitore di tensione collegato al pin `ADC_BAT` (GPIO 4), per calcolarne la percentuale residua.

1.4 BMA400 e CJMCU-400

L'accelerometro utilizzato è un BMA400 [4] integrato sulla *breakout board* CJMCU-400. Il sensore viene interrogato per rilevare il movimento, consentendo l'implementazione di un contapassi utile al monitoraggio dell'attività motoria dell'animale.

2 Firmware e logica di funzionamento

Il firmware, sviluppato in ambiente Arduino per il *System on Chip* (SoC) ESP32-S3, gestisce l'acquisizione dei dati dai sensori, il controllo del modem LTE e l'ottimizzazione energetica tramite una macchina a stati finiti.

2.1 Gestione della persistenza e hot start

Nel tracciamento satellitare, il *Time To First Fix* (TTFF) rappresenta il tempo impiegato dal modulo GNSS, a partire dalla sua accensione, per acquisire i dati satellitari e calcolare la prima coordinata valida. Trattandosi di un processo che richiede di mantenere il modulo radio attivo a lungo, costituisce la fase a maggior consumo.

Per massimizzare la durata della batteria, il dispositivo utilizza il *deep sleep*, uno stato di ibernazione in cui l'ESP32 spegne la CPU e la memoria principale. L'unico componente che rimane acceso è il modulo *Real-Time Clock* (RTC) dell'ESP32, dotato di una piccola memoria dedicata. Per ridurre il TTFF, il firmware sfrutta quest'area per preservare i dati critici durante i cicli di *deep sleep*. Dove, le variabili come l'ultima posizione valida (`lastLat`, `lastLon`) e i timestamp (`lastGpsDate`, `lastGpsTime`) vengono dichiarate con l'attributo `RTC_DATA_ATTR`, permettendo al sistema di ritrovarle intatte al risveglio.

Al risveglio del dispositivo, la funzione `iniettaGps()` esegue le seguenti operazioni:

- Recupera le coordinate geografiche e temporali salvate nel ciclo precedente;
- Inietta la posizione nota nel modulo GNSS tramite il comando `AT+CGNSSPOS`;
- Sincronizza l'orologio interno del modem con il comando `AT+CGNSSTIME`.

Questo meccanismo abilita la modalità *hot start*, che permette di agganciare il segnale in pochi secondi, abbattendo drasticamente il consumo energetico.

2.2 Protocollo di comunicazione HTTPS

La trasmissione dei dati al *backend* PocketBase è affidata a una serie di comandi AT inviati via seriale al modem SIM7670G. Il firmware incapsula le informazioni in un oggetto JSON strutturato, trasmesso tramite una richiesta HTTPS POST. Il *payload* include:

- `board_id`: un identificativo univoco derivato dall'IMEI del modem;
- `geo`: un oggetto con latitudine e longitudine per la localizzazione geografica;
- `battery / battery_percent / charging`: dati sullo stato di carica della batteria;
- `steps`: conteggio dei passi rilevato dall'accelerometro nella sessione corrente;
- `sleep`: flag booleano che segnala al server l'imminente entrata in stato di inattività;
- `trip`: flag booleano che segnala al server che l'animale si trova su un veicolo in movimento. Quando attivo, ignora le regole del *geofencing* e sospende il *deep sleep* per garantire la continuità del tracciamento.

La sequenza di invio utilizza i comandi `AT+HTTPINIT` per inizializzare il servizio HTTP del modem, `AT+HTTPPARA` per definire i parametri della connessione, `AT+HTTPDATA` per specificare la dimensione del JSON, e infine `AT+HTTPACTION=1` per avviare la trasmissione. Al termine di ogni invio, `AT+HTTPTERM` chiude la sessione per liberare le risorse del modem.

2.3 Risparmio energetico

Per massimizzare l'autonomia, la gestione energetica viene modulata in base alla rilevazione del movimento e allo stato di viaggio:

1. **Rilevamento attività:** il firmware monitora costantemente il numero di passi, azzerando a ogni nuovo movimento un timer di inattività (`lastActivityTime`).
2. **Modalità viaggio:** se il flag `trip` è attivo (velocità GPS > 10 km/h), il *deep sleep* viene disabilitato indipendentemente dall'inattività rilevata dall'accelerometro, garantendo la trasmissione dei dati a intervalli regolari durante il viaggio.
3. **Ibernazione:** se non viene rilevata attività per un tempo superiore a `SLEEP_TIMEOUT` (30 secondi) e il flag `trip` non è attivo, il sistema invia un ultimo pacchetto dati aggiornando il flag `sleep` a `true`, entrando in *deep sleep*. Vengono quindi spenti i moduli radio tramite i comandi `AT+CGNSSPWR=0` e `AT+CPOWD=1`. Il risveglio è gestito tramite interrupt hardware sul pin `GPIO_5`, generato dall'accelerometro solo al rilevamento di un nuovo movimento.

Inoltre, il firmware integra specifiche strategie di controllo energetico volte a eliminare i consumi statici e a prevenire alterazioni nelle letture dei sensori:

- **Controllo ADC:** il circuito del partitore di tensione per la batteria viene attivato solo durante la lettura tramite il pin `BAT_ADC_EN`, garantendo la precisione del voltaggio acquisito dal pin `ADC_BAT`. Il valore grezzo viene mediato su 10 campionamenti consecutivi e scalato con fattore pari a 2 per compensare il partitore. La percentuale di carica è calcolata per interpolazione lineare nell'intervallo compreso tra 3,4 V e 4,2 V, corrispondenti rispettivamente a 0% e a 100% di carica.
- **Stato USB:** dal registro hardware `USB_SERIAL_JTAG_EP1_CONF_REG` dell'ESP32, il sistema rileva se il dispositivo è collegato a una fonte di ricarica USB, impostando dinamicamente il campo `charging` nel pacchetto dati. In stato di ricarica, la lettura della batteria viene interrotta per evitare alterazioni dovute alla tensione in ingresso e vengono trasmessi semplicemente gli ultimi dati validi salvati nella memoria RTC.

2.4 Acquisizione GPS e timeout

Per acquisire la posizione, il firmware interroga il modulo GNSS tramite il comando `AT+CGNSSINFO`. La risposta viene analizzata campo per campo, estraendo latitudine, longitudine e velocità in formato decimale. Questo evita al microcontrollore il dover decodificare il protocollo standard NMEA, che esprime le coordinate in gradi e minuti e richiederebbe calcoli matematici aggiuntivi per ricavare il formato decimale utilizzato dalle mappe moderne per il **backend**. Il firmware attende fino al `GPS_TIMEOUT` (180 secondi) per ottenere una posizione valida; superato tale limite senza successo, il ciclo corrente viene saltato e il sistema ritenta al loop successivo. Ottenute invece le coordinate, queste e il timestamp vengono salvati in memoria RTC per abilitare l'*hot start* al ciclo successivo.

2.5 Connessione alla rete LTE

All'avvio, attraverso la funzione `connectToNetworkFast()`, il firmware tenta di registrarsi alla rete LTE. Vengono configurati la modalità radio (`AT+CNMP=38` per LTE), il profilo APN dell'operatore e l'attivazione del contesto dati con `AT+CNACT=0,1`. La registrazione viene verificata tramite interrogazioni periodiche con il comando `AT+CEREG?`, controllando che il modem restituisca i codici di stato `0,1` (registrato in rete home) o `0,5` (roaming). Se entro il `NET_TIMEOUT` (60 secondi) la registrazione non viene completata, il dispositivo entra direttamente in *deep sleep* per evitare consumi inutili in assenza di copertura.

3 Backend e database

La gestione della persistenza e della modellazione dei dati è basata su PocketBase [5], una soluzione open-source strutturata su SQLite scelta per la sua reattività, il ridotto consumo di risorse e il supporto di un'interfaccia API RESTful. La logica applicativa lato server e l'integrazione dei flussi di dati si sviluppano estendendo il nucleo del framework direttamente nel file principale di runtime `main.pb.js`, che gestisce sia gli eventi reattivi, noti come *hooks*, sia i servizi temporali pianificati, definiti come *cron jobs*. Poiché il runtime JavaScript di PocketBase si basa sull'interprete Goja incorporato in Go, l'intera esecuzione del codice avviene in modalità *single-threaded*, condividendo il medesimo spazio d'esecuzione globale e lo stesso ciclo di eventi del server.

3.1 Architettura di hosting e affidabilità del sistema

L'infrastruttura operativa di backend è stata strutturata per garantire disponibilità dei dati, tolleranza ai guasti e monitoraggio continuo durante la fase di prototipazione e test:

- **Esposizione in rete:** il server PocketBase è ospitato su una *workstation* locale. Data la necessità di connettersi al backend sia dal dispositivo (tramite rete cellulare LTE) sia dall'applicazione mobile (tramite rete dati/Wi-Fi), le restrizioni imposte dal NAT e l'assenza di un IP pubblico statico sono state superate mediante l'impiego di ngrok [6]. Questa tecnologia di *tunneling* crea un canale sicuro HTTPS fungendo da *reverse proxy*, reindirizzando il traffico in ingresso verso l'ambiente locale e garantendo l'accessibilità dei servizi in tempo reale.
- **Strategia di backup:** per garantire la resilienza dei dati e prevenire perdite accidentali dovute a guasti del server locale, è stato strutturato un sistema di backup automatizzato tramite rclone [7], uno strumento a riga di comando per la gestione del cloud storage. Il processo è configurato per eseguire la sincronizzazione incrementale della directory dei dati di PocketBase verso un *bucket remoto* su Backblaze B2 [8], attivato tramite un *cron job* a cadenza mensile.
- **Monitoraggio esterno:** l'accessibilità e lo stato di salute del sistema sono monitorati esternamente tramite UptimeRobot [9]. Poiché l'accessibilità del backend dipende dalla stabilità del tunnel ngrok, il servizio interroga l'endpoint HTTPS a intervalli regolari. Questo permette di rilevare tempestivamente eventuali anomalie di rete, inviando notifiche immediate in caso di interruzione del tunnel o spegnimento del server locale.

3.2 Modellazione dei dati e API rules

I dati inviati dal dispositivo transitano inizialmente nella collezione di staging `data_sent_raw`, fungendo da *buffer* iniziale per i dati non ancora elaborati, separando la fase di ricezione dalla successiva validazione. Il singolo record è strutturato secondo i seguenti campi:

- `timestamp`: riferimento temporale hardware dell'acquisizione;
- `lat / lon`: coordinate geografiche acquisite dal modulo GNSS;
- `battery / battery_percent`: stato di carica dell'accumulatore espresso in voltaggio e percentuale;
- `charging`: variabile booleana indicante se la batteria è in corso di ricarica;
- `steps`: conteggio lineare dei passi rilevato dall'accelerometro;
- `sleep`: flag booleano indicante se il dispositivo è in stato di *deep sleep*;
- `trip`: flag booleano indicante se l'animale si trova su un veicolo in movimento.

L'accesso ai dati avviene esclusivamente tramite le API RESTful esposte da PocketBase. In questo modo il database rimane totalmente indipendente dall'applicazione mobile sviluppata in Flutter, che si limita a richiamare le API per visualizzare i dati.

3.3 Pulizia e normalizzazione (Server-side Hooks)

L'elaborazione dei messaggi IoT segue un pattern architetturale di *Staging-and-Purge* gestito lato server tramite i *JavaScript Event Hooks* nativi del sistema. L'hook `onRecordAfterCreateSuccess` si attiva immediatamente dopo il corretto inserimento di ogni pacchetto grezzo nella collezione `data_sent_raw`. Per ottimizzare le prestazioni ed eliminare letture ridondanti, il sistema effettua un unico accesso iniziale per recuperare la configurazione del dispositivo (`getBoardRecord`) all'inizio dell'esecuzione dell'hook, distribuendo il record ottenuto come parametro a tutte le funzioni della pipeline. L'intero flusso di elaborazione si articola in quattro fasi sequenziali:

1. **Batteria:** salva il record in una collezione dedicata `battery_data` e verifica se inviare notifiche push per segnalare livelli critici di batteria.
2. **Activity Tracking:** delega l'analisi della telemetria al modulo `activity\_manager.js`, che aggiorna la macchina a stati e gestisce la persistenza delle sessioni.
3. **Posizioni:** salva le coordinate GPS nella collezione `positions`, associandole all'attività identificata al punto precedente, a condizione che i dati GNSS risultino attendibili e l'attività sia considerata valida.
4. **Eliminazione dato grezzo:** rimuove permanentemente il record originario da `data_sent_raw` all'interno del blocco `finally` dell'hook. Questo costrutto non serve a garantire l'atomicità transazionale (poiché l'evento scatta a inserimento già avvenuto), ma funge da meccanismo di sicurezza per l'ottimizzazione dello spazio. Infatti, assicura la cancellazione fisica del record di staging e previene la crescita incontrollata del database SQLite, anche qualora una delle funzioni precedenti della pipeline dovesse lanciare un'eccezione non gestita a runtime.

3.4 Macchina a stati delle attività

Il modulo `activity_manager.js` implementa una macchina a stati che classifica il movimento e la posizione dell'animale, in funzione dell'area di **geofence**, dell'**allarme** impostato dall'utente e della **velocità** dell'animale. Ogni stato operativo assume una diversa nomenclatura e conserva lo stesso significato logico, a seconda che il dispositivo sia *attivo* o in *sleep* (*deep sleep*), mantenendo una corrispondenza biunivoca riassunta nella Tabella 1.

Attivo	Sleep	Allarme	Geofence	Descrizione posizione
i (inside)	d	True/False	True	All'interno dell'area sicura.
v (vehicle)	a	False	False	In viaggio su veicolo.
r (robbed)	q	True	False	Rischio di furto su veicolo.
w (walk)	z	False	False	In passeggiata.
s (search)	p	True	False	Rischio di fuga.

Tabella 1: stati di posizionamento dell'animale.

A livello di archiviazione, l'ingresso in uno stato di sleep chiude la sessione corrente, impostando il flag `is_active = false`. Tuttavia, al momento di elaborare i nuovi dati, il sistema ha bisogno di recuperare il contesto originario dell'animale. Per questo motivo, prima di applicare la logica di classificazione, la funzione `computeStatus` normalizza

internamente lo stato di *sleep*, riconducendolo al suo equivalente attivo tramite il dizionario `SLEEP_TO_ACTIVE` (es. "d" → "i"). Questo meccanismo garantisce la continuità dell'analisi anche dopo lunghe fasi di inattività hardware.

Per evitare la frammentazione dei dati e ottimizzare le query storiche, il sistema applica un principio di **continuità temporale**. Finché l'animale permane nel medesimo stato, il sistema non crea nuovi record nel database, ma si limita ad aggiornare il timestamp finale (`end_time`) e a incrementare il conteggio dei passi (`total_steps`). L'inserimento di una nuova riga nel database, con la contestuale chiusura della precedente, avviene esclusivamente in presenza di un'effettiva transizione di stato. Una transizione tra "v" e "r", indotta dal cambio manuale del flag di allarme da parte dell'utente, forza la chiusura e la riapertura immediata della sessione, permettendo di storicizzare l'evento e di adeguare istantaneamente la priorità delle notifiche. Le transizioni tra gli stati sono governate da un algoritmo a priorità decrescente:

1. **Priorità 1 (massima):** se il sensore rileva che l'animale è in viaggio (`trip = true`) e il contatore dei passi è nullo (`steps == 0`), lo stato viene forzato a "v" o "r" in base all'allarme. Questa condizione funge da filtro per i falsi positivi, ignorando eventuali vibrazioni o rumori dell'accelerometro.
2. **Priorità 2:** se l'animale non è in viaggio (`trip = false`) ma non sono stati ancora rilevati passi (`steps == 0`), il sistema applica una memoria di stato. Nel dettaglio, assume che il veicolo sia momentaneamente fermo (es. a un semaforo o in sosta con l'animale a bordo) e preserva lo stato "v" o "r" corrente. Il ricalcolo della posizione si attiva solo se `steps > 0`, ovvero al primo movimento dell'animale.
3. **Priorità 3:** se l'animale è nell'area di geofence, lo stato viene impostato a "i".
4. **Priorità 4 (minima):** se l'animale si trova all'esterno dell'area sicura, il sistema valuta il *flag* di allarme per determinare lo stato finale, impostando "w" (passeggiata) in caso di allarme disattivato, oppure "s" (ricerca) in caso di allarme attivato.

3.5 Lifecycle del deep sleep e servizi temporali

3.5.1 Gestione del risveglio e ripristino dello stato

Al termine di un ciclo di *deep sleep*, il dispositivo riprende le trasmissioni inviando un nuovo pacchetto dati. In quel momento non è presente alcuna sessione attiva sul database, poiché quest'ultima viene forzatamente chiusa al passaggio in modalità a basso consumo. Per garantire la continuità logica, il sistema interroga lo storico escludendo eventuali record anomali e recupera l'ultima sessione valida archiviata (`recentClosed`). Questo passaggio è fondamentale per estrarre lo stato precedente (`prevStatus`), parametro necessario per la corretta esecuzione della funzione `computeStatus`, che gestisce il risveglio secondo due scenari:

- **Risveglio nello stesso stato:** si verifica quando lo stato rilevato al risveglio coincide con l'equivalente *attivo* dell'ultimo stato di *sleep*. Ad esempio, se il dispositivo si era sospeso in area sicura (stato d, equivalente a i) e al risveglio rileva che il nuovo stato è ancora i, la sessione pregressa viene semplicemente riaperta (`is_active = true`). Questo approccio evita la frammentazione dei record, unificando le attività separate dalla fase di sospensione.

- **Risveglio con cambio di stato:** si verifica quando il nuovo stato rilevato non coincide con l'equivalente *attivo* del precedente stato di *sleep*. In questo caso, il sistema separa nettamente le due fasi, storicizza definitivamente la sessione pregressa, normalizzandone lo stato nella rispettiva controparte *attiva* (es. lo stato *d* viene convertito in *i*) per garantire la coerenza storica. Subito dopo, istanzia un nuovo record nel database per il nuovo stato.

Per ottimizzare l'efficienza del thread unico di esecuzione, il modulo implementa un **filtraggio delle notifiche spurie** (*STEP 3 anticipato*): se viene ricevuto un pacchetto di quiete (`sleep = true`) in assenza di una sessione attiva sul database, l'inserimento viene intercettato e scartato preventivamente a livello logico, impedendo la proliferazione di record ridondanti e l'invio di catene di notifiche push duplicate all'utente.

3.5.2 Watchdog e split notturno

La coerenza temporale delle sessioni di attività e l'allineamento dello storico sono garantiti da due *cron job*, registrati all'avvio all'interno del *thread* principale dell'applicazione:

- **Watchdog:** ogni minuto interroga il database, per individuare e interrompere le sessioni attive da cui non si ricevono aggiornamenti per oltre 10 minuti, tipicamente a seguito di malfunzionamenti o perdite di connettività LTE. Per preservare lo storico, il modulo chiude forzatamente il record (`is_active = false`), lo marca come anomalo (`anomaly = true`) e notifica tempestivamente l'utente. Da questa procedura di sicurezza sono escluse per *design* le attività in *sleep*, la cui inattività prolungata rientra nel normale funzionamento.
- **Split notturno:** suddivide le sessioni operative che si protraggono a cavallo di due giornate (esattamente alle 23:59 ora italiana), al fine di non alterare le statistiche grafiche giornaliere visibili all'utente. Poiché i *timestamp* sono memorizzati nativamente in UTC, il sistema gestisce dinamicamente l'alternanza tra ora solare (CET, 22:59:59 UTC) e ora legale (CEST, 21:59:59 UTC) tramite la funzione `getItalyTime()`, evitando l'impiego di valori di fuso orario statici. Il *cron job* è pertanto schedato per avviarsi in entrambi i possibili orari critici (`59 21,22 * * *`). Sarà poi un controllo interno, basato sul fuso orario attivo, a bloccare l'esecuzione nel caso in cui l'orario non corrisponda a quello effettivo. In fase di elaborazione, le sessioni in stato di *sleep* vengono preventivamente riattivate per garantire l'integrità dello storico, mentre le occorrenze già marcate come anomale (`anomaly = true`) vengono ignorate, impedendone la duplicazione nel giorno successivo.

4 Frontend e applicazione mobile

L'applicazione mobile è stata sviluppata in *Flutter* [10], un framework cross-platform che consente di produrre un'unica base di codice per Android e iOS. I test sono stati condotti **esclusivamente su ambiente Android** a causa di vincoli tecnici. Nello specifico, l'assenza di postazioni di lavoro con macOS ha impedito la compilazione per l'ecosistema Apple, a cui si aggiunge la mancanza di una licenza per l'*Apple Developer Program*, necessaria per superare il limite di persistenza di sette giorni previsto per i profili gratuiti su dispositivo fisico. La comunicazione con il backend avviene tramite il PocketBase SDK ufficiale per *Dart*, che gestisce le chiamate REST e i meccanismi di autenticazione.

4.1 Architettura a livelli

Il codice è strutturato in quattro livelli principali, per separare le diverse responsabilità:

- **Repositories:** accesso ai dati remoti tramite PocketBase SDK, fornendo alle schermate dati in tempo reale e pronti all'uso.
- **Models:** definiscono la struttura dei dati convertendo i record JSON del backend in oggetti Dart.
- **Services:** gestiscono le funzionalità di sistema e le integrazioni esterne (autenticazione, mappe, tracciamento GPS e notifiche).
- **Screens:** interfaccia utente che si limita a richiedere i dati ai repository e a reagire ai loro aggiornamenti.

Questa struttura garantisce un codice ordinato e facile da aggiornare, mantenendo la logica di funzionamento ben separata dall'interfaccia grafica.

4.2 Autenticazione e persistenza della sessione

Per permettere all'utente di restare collegato in modo permanente e sicuro, abbiamo esteso l'`AuthStore` nativo di PocketBase, che normalmente salva i dati solo temporaneamente in memoria. L'autenticazione è quindi gestita tramite un `SecureAuthStore` personalizzato che agisce come una *cassaforte digitale*, riceve il token (JSON Web Token) dal server al momento del login e lo archivia in forma cifrata all'interno del dispositivo tramite `flutter_secure_storage`. Questo sistema sfrutta i sistemi di protezione hardware del dispositivo (Keystore su Android e Keychain su iOS) per impedire accessi non autorizzati.

Al riavvio dell'app, il token viene riletto dallo storage sicuro e viene tentato un refresh automatico della sessione. In caso di successo, l'utente accede direttamente alla schermata principale senza dover reinserire le credenziali; in caso di token scaduto o non valido, lo store viene svuotato e l'utente viene reindirizzato alla schermata di login.

4.3 Splash screen e caricamento parallelo

La schermata di avvio svolge un ruolo attivo nel precaricare i dati necessari alla home page. Una volta verificata l'autenticazione, vengono avviate in parallelo, le seguenti operazioni:

- recupero del profilo utente e dello stato dell'allarme;
- recupero dell'ultima posizione GPS registrata dal dispositivo;
- recupero delle attività del giorno corrente filtrate per `board_id`.

Il `board_id`, identificativo univoco della board LilyGo associata a un dispositivo per utente, viene risolto dinamicamente interrogando la collezione `boards` sul backend tramite repository. Questo permette all'app di trovare il dispositivo corretto automaticamente, evitando di dover inserire il codice manualmente nel software.

4.4 Comunicazione in tempo reale

Completata la fase iniziale di caricamento dati, l'app stabilisce una **connessione persistente** al database, passando da una visualizzazione statica ad un monitoraggio dinamico.

Per le funzionalità che richiedono aggiornamenti continui come la posizione GPS, l'app sfrutta il meccanismo di *subscribe* offerto da PocketBase, basato su **Server-Sent Events**. I repository (come **PositionsRepository**) espongono uno **Stream** tipizzato (ad esempio **Stream<Positions>**) a cui le schermate si sottoscrivono tramite una **StreamSubscription**.

In questo modo, ogni nuovo record inserito nel database viene notificato istantaneamente all'app. L'interfaccia utente si aggiorna dinamicamente evitando la tecnica del *polling*, che costringerebbe l'app a inviare continue richieste al server, ed eliminando la necessità di qualsiasi ricaricamento manuale della pagina da parte dell'utente.

Per garantire l'efficienza dell'applicativo e prevenire *memory leak*, ogni sottoscrizione viene rigorosamente interrotta tramite il comando **cancel()** nel metodo **dispose()** quando l'utente naviga verso un'altra schermata. Questa accortezza evita di mantenere attive connessioni inutili in background, ottimizzando l'uso della batteria e del traffico dati.

4.5 Geofencing e geocoding

La logica di geofencing è implementata anche dal lato client tramite algoritmo *Ray-Casting*. Dato un punto GPS e un poligono definito da una lista di vertici, l'algoritmo traccia un raggio orizzontale dal punto verso l'infinito e conta le intersezioni con i lati del poligono, se il numero è dispari indica che il punto è interno al poligono, altrimenti è esterno. Questa implementazione sul dispositivo evita un sovraccarico di rete verso il server e permette di gestire le funzionalità visive in tempo reale, in totale autonomia dal *backend*.

Le zone vengono recuperate dalla collezione **geofences** su PocketBase, dove ogni record memorizza il nome della zona, i dati dell'indirizzo e la lista dei vertici del poligono. Solo le zone con il flag **is_active** impostato a **true** vengono considerate nel calcolo. Il risultato della verifica è una stringa con il nome della zona di appartenenza, oppure *"Fuori zona sicura"* se il punto non rientra in nessuna area attiva.

L'app integra un servizio di **geocoding** basato sulle API di *Nominatim* [11], un motore di ricerca che interroga il database geografico open-source di OpenStreetMap per consentire:

- **Reverse Geocoding**: tradurre le coordinate GPS in un indirizzo testuale completo (Via, Civico, CAP e Città) per facilitare la lettura all'utente;
- **Forward Geocoding**: consentire all'utente di creare nuove zone inserendo un indirizzo testuale, che il sistema converte automaticamente in coordinate geografiche per posizionare correttamente il poligono sulla mappa.

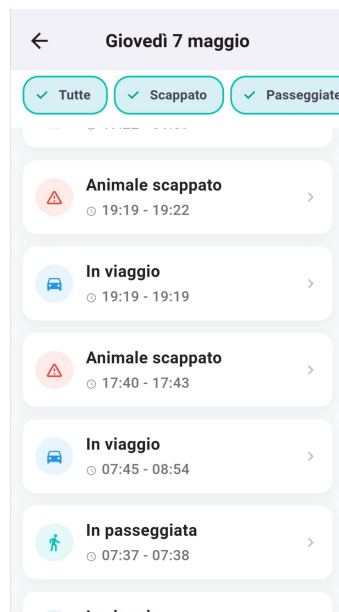
4.6 Schermate e funzionalità principali

4.6.1 Analisi delle attività

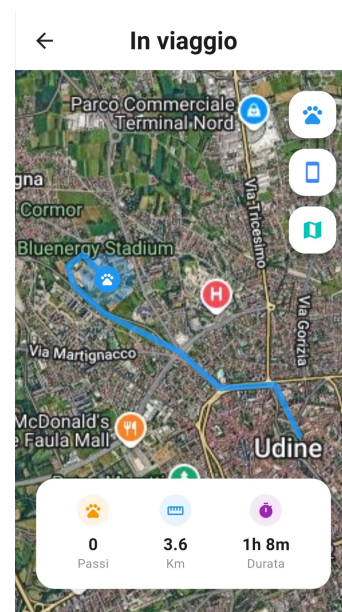
L'architettura dell'applicazione ruota attorno alla dashboard principale in **home.dart**, che funge da centro di controllo dinamico per il monitoraggio dell'animale, mostrandone l'ultima posizione rilevata. Un'altra caratteristica importante della pagina è l'interruttore dell'allarme. Se il sistema rileva che l'animale è in stato di fuga, l'interfaccia attiva automaticamente un blocco di sicurezza per impedirne la disattivazione. Inoltre, poiché lo stato dell'allarme si basa sulla variabile condivisa **_isUpdatingAlarm**, l'integrità dell'operazione è garantita da un **mutex** che disabilita temporaneamente l'interruttore durante

la sincronizzazione asincrona con il database. Infine, la schermata raccoglie le statistiche quotidiane come passi, chilometri e tempo di attività dell'animale, permettendo la consultazione dei dati storici tramite un selettore di data.

La visualizzazione dei dettagli delle attività di una giornata prosegue in `daily_recap.dart`, che raccoglie e organizza in ordine cronologico ogni monitoraggio visualizzato nella dashboard, con la possibilità di filtrare per specifiche attività. Selezionando una determinata sessione dal riepilogo, si accede alla schermata `activity_details.dart`, che mostra il tracciato specifico del percorso tramite una scia continua sulla mappa. Il sistema utilizza la funzione `CameraFit` per centrare l'inquadratura all'apertura, garantendo l'immediata visibilità dell'intero tragitto. Inoltre, la schermata ripropone le statistiche di passi, chilometri e durata, ricalcolate specificamente per l'attività selezionata.



(a) Visualizzazione di tutte le attività.



(b) Dettagli di un'attività.

4.6.2 Mappe in tempo reale

L'app utilizza mappe interattive sviluppate con `flutter_map`, permettendo di localizzare utente e animale in una visuale satellitare o stradale. La libreria viene ottimizzata tramite l'uso dei tile di `OpenStreetMap`, ovvero piccoli tasselli d'immagine caricati dinamicamente solo per l'area inquadrata, riducendo drasticamente il consumo di dati. Per preservare la batteria dello smartphone, le posizioni si aggiornano in tempo reale solo quando c'è un vero movimento, inoltre l'utente viene localizzato solo se ha dato il permesso.

Il sistema di controllo geografico si articola in due schermate principali che si scambiano dinamicamente in base allo stato dell'allarme. Quando l'allarme è disattivato, l'utente accede al sistema di `geofencing.dart`, un robusto strumento di sicurezza preventiva che permette di definire e gestire i confini virtuali rappresentati come poligoni sulla mappa.

All'attivazione dell'allarme, la schermata `geofencing.dart` passa a `tracking.dart`. In questa fase, il sistema adotta un approccio proattivo di sicurezza implementando un'interfaccia adattiva che cambia tra una modalità di monitoraggio e una di "inseguimento" qualora l'animale lasci la sua zona sicura attiva. Sulla mappa inoltre viene renderizzata

una scia storica delle ultime posizioni registrate, fornendo un percorso visivo immediato che aiuta a comprendere la direzione del movimento recente dell'animale.

4.6.3 Gestione delle zone sicure

Dopo l'inserimento di un indirizzo (manuale o GPS) o la selezione di una zona dalla pagina di geofencing, la creazione e modifica delle aree avviene tramite `polygon_editor.dart`. L'utente può definire il perimetro toccando lo schermo per aggiungere i vertici. L'interfaccia è progettata per essere estremamente flessibile, consentendo all'utente non solo di aggiungere o annullare i vertici, ma anche di rifinirne la forma trascinando i punti già posizionati. Un'area può essere creata solo se presenta un minimo di tre vertici per formare una figura geometrica chiusa e rispetta due controlli di validità:

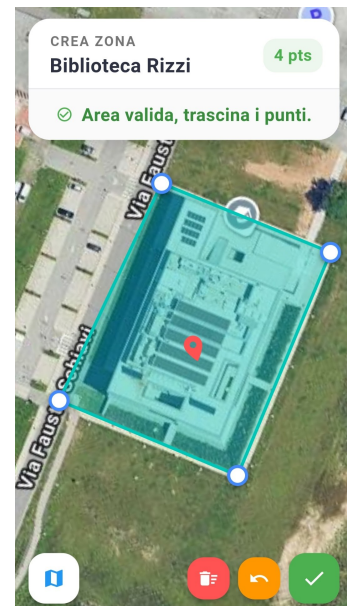
- Il **baricentro dell'area** non deve essere troppo distante dall'indirizzo selezionato, impedendo così la creazione di zone a più di 50 metri dal segnaposto reale.
- Non deve esserci alcuna **sovrapposizione con zone già esistenti** al fine di garantire l'univocità dei perimetri ed evitare conflitti logici durante il tracciamento.



(a) Scelta dell'inserimento.



(b) Compilazione indirizzo.



(c) Disegno dell'area.

5 Sistema di notifiche push

L'implementazione di un sistema di notifiche push è fondamentale per un dispositivo di tracciamento animali, in quanto permette di avvisare tempestivamente l'utente in caso di pericoli (uscita dal perimetro sicuro) o problemi tecnici (batteria scarica).

5.1 Analisi dei vincoli tecnologici

Durante la fase di sviluppo, è emerso un vincolo tecnico significativo riguardante l'integrazione tra il backend PocketBase e l'SDK ufficiale di Firebase. PocketBase è scritto in linguaggio Go e utilizza **Goja** come motore di esecuzione JavaScript per la gestione della

logica server-side (hooks). Goja è un'implementazione di ECMAScript 5.1 che presenta limitazioni strutturali rispetto a un ambiente Node.js [12] standard:

- **Mancanza di moduli nativi:** Goja non supporta i moduli core di Node.js (come `fs`, `crypto` o `net`), indispensabili per il funzionamento dell'SDK di Firebase.
- **Runtime isolato:** l'SDK `firebase-admin` [13] richiede un ambiente di esecuzione completo per gestire l'autenticazione tramite Service Account e le connessioni persistenti ai server Google.

Per superare queste limitazioni, è stata adottata un'architettura **Bridge**, separando la logica applicativa dall'invio fisico dei messaggi.

5.2 Architettura del FCM Bridge

Il sistema si basa su un microservizio esterno sviluppato in Node.js [12], denominato *FCM Bridge*. L'interazione avviene secondo il seguente flusso:

1. **PocketBase (Goja):** rileva un evento (es. batteria scarica) e invia una richiesta HTTP POST al bridge tramite il modulo interno `$http.send`.
2. **FCM Bridge (Node.js):** riceve la richiesta, carica il Service Account di Firebase e inoltra la notifica tramite l'SDK ufficiale [13].
3. **Invio notifica:** il bridge restituisce lo stato della consegna. In caso di errore 404 (token non registrato, app disinstallata) o 400 (token malformato), il backend provvede alla rimozione automatica del token dal profilo utente. I token FCM sono memorizzati come array JSON per supportare più dispositivi per singolo utente.

5.3 Logica di invio e trigger

Le notifiche vengono scatenate dai seguenti eventi, ciascuno con logica contestuale per evitare invii ridondanti nella Tabella 2.

Evento	Condizione di trigger	Nota
Uscito dalla zona	$i \rightarrow s$ oppure $i \rightarrow w$	Dipende dallo stato allarme
Rientro in zona	$s \rightarrow i$ oppure $w \rightarrow i$	Presenza di geofence
Allarme in passeggiata	$w \rightarrow s$	Attivazione allarme
Inizio viaggio confermato	$qualsiasi \rightarrow v/r$	Dipende dallo stato allarme
Fine viaggio in zona	$v/r \rightarrow i$	Rientro in geofence
Fine viaggio fuori zona	$v/r \rightarrow s$	Se allarme attivo
Fine viaggio fuori zona	$v/r \rightarrow w$	Se allarme spento
Nessuna geofence	$i \text{ o } s \rightarrow w$	Assenza di geofence
Batteria bassa	$\leq 20\%$	Solo al cambio di soglia
Batteria critica	$\leq 10\%$	Solo al cambio di soglia
Batteria in carica	<code>charging</code> passa a <code>true</code>	Solo al cambio di stato
Watchdog	Silenzio > 10 minuti	Chiusura automatica

Tabella 2: Trigger del sistema di notifiche push

5.4 Gestione permessi

Per abilitare la ricezione delle notifiche lato client, è stato fondamentale integrare il file di configurazione `google-services.json`, generato direttamente dalla console di Firebase e inserito all'interno della directory `android/app` del codice sorgente. Questo file contiene le chiavi API, l'identificativo del progetto (`Project ID`) e i dettagli di configurazione necessari per autenticare in modo sicuro l'applicazione mobile con i servizi Google, senza esporre le credenziali nel codice in chiaro. Durante la fase di build, il *plugin Gradle* di Google Services, che si occupa di mappare i parametri di configurazione nel codice Android, processa questo file per inizializzare automaticamente l'SDK di Firebase all'avvio dell'app, abilitando così la connessione sicura a FCM e la generazione del token univoco del dispositivo. Quest'ultimo viene dapprima salvato localmente e poi sincronizzato con il profilo utente su PocketBase all'interno del campo `tokenFCM`. Prima di ogni salvataggio, viene effettuato un controllo di duplicazione che verifica se il *token* non sia già presente, evitando così la ricezione di notifiche ridondanti.

6 Vincoli di sistema

Nonostante i risultati ottenuti, il sistema presenta alcune limitazioni:

- **Dipendenza dalla rete LTE:** senza copertura la trasmissione dati non è garantita.
- **Prestazioni indoor:** il modulo GNSS risulta poco affidabile in ambienti chiusi.
- **Backend locale:** l'utilizzo di PocketBase su macchina locale, esposto tramite ngrok, introduce dipendenze da servizi esterni e limita la scalabilità.
- **Race condition sui pacchetti:** in caso di ricezione quasi simultanea di due pacchetti dallo stesso dispositivo, entrambi potrebbero leggere lo stesso record `activity` prima del completamento della prima scrittura. Gli scenari più critici sono due:
 - Se entrambi i pacchetti rilevano un cambio di stato, il secondo potrebbe sovrascrivere un record già chiuso dal primo, generando inconsistenze strutturali.
 - Se entrambi i pacchetti riattivano lo stesso record `sleep`, potrebbero creare due sessioni attive contemporaneamente, violando l'invariante del sistema.

L'impossibilità di gestire transazioni atomiche sui singoli record tramite le API di PocketBase rende questo scenario non eliminabile lato applicativo. Tuttavia, la frequenza di invio (ogni 30 s circa) rende l'evento estremamente raro nella pratica.

- **Fallimento nella geolocalizzazione:** l'eventuale mancato rilevamento di indirizzi, solitamente nei centri abitati più piccoli, non è imputabile al codice, bensì a un limite legato alla densità dei metadati geografici. Poiché `OpenStreetMap` si basa su contributi volontari, l'assenza di tag specifici (come i numeri civici o il CAP associato alla via) rende inefficaci le query strutturate inviate a `Nominatim`. Quest'ultimo, richiedendo una corrispondenza rigida ed esatta di tutti i parametri forniti, non è in grado di compensare l'incompletezza dei dati.

7 Sviluppi futuri e prospettive del progetto

Durante lo sviluppo sono emerse diverse opportunità di scalabilità del progetto, illustrate di seguito tramite alcune proposte di ottimizzazione ed estensione delle funzionalità.

7.1 Evoluzione e scalabilità del backend

L'evoluzione dall'attuale prototipo locale a una soluzione più robusta ed efficace richiederà la migrazione dell'intero backend su un'infrastruttura Cloud dedicata, adottando soluzioni PaaS o VPS (come AWS, Google Cloud o DigitalOcean). Questa evoluzione consentirà di superare i vincoli attuali attraverso i seguenti interventi strutturali:

- **Assegnazione di dominio e sicurezza TLS:** sostituendo il tunnel temporaneo `ngrok` con un dominio di secondo livello proprietario, protetto da certificati SSL/TLS (es. `Let's Encrypt`), garantirà una connessione HTTPS/WSS, stabile e priva di interruzioni di sessione o limiti di traffico.
- **Risoluzione delle race condition:** per eliminare il rischio di collisioni causato dall'arrivo simultaneo di pacchetti, si prevede l'introduzione di un middleware di messaggistica asincrona, come `Redis` o `RabbitMQ`, tra l'endpoint di ricezione e il database. Questo livello fungerà da coda FIFO (*First-In, First-Out*), mettendo in fila i pacchetti provenienti dallo stesso dispositivo per elaborarli rigorosamente uno alla volta, garantendo così la coerenza dei dati nel database.
- **Migrazione del DBMS:** qualora il volume di dispositivi connessi diventasse troppo elevato per le capacità di SQLite, sarà possibile migrare verso un database più robusto (come `PostgreSQL`), progettato appositamente per gestire un grande volume di scritture simultanee senza bloccarsi.

7.2 Evoluzione e costruzione hardware

L'attuale prototipo è stato fondamentale per testare il software, ma richiederà un'ottimizzazione dell'hardware per diventare un dispositivo pratico da far indossare all'animale. Lo sviluppo futuro si articolerà su tre assi principali:

- **Sviluppo di un PCB custom:** progettare un'unica scheda su misura (es. `KiCad`) che integri microcontrollore, modem LTE e modulo GNSS. In questo modo si ridurranno drasticamente le dimensioni e il peso del dispositivo sul collare dell'animale.
- **Ottimizzazione del power management:** l'integrazione di un *PMIC* e di un chip fuel gauge (es. `MAX17043`) permetterà di ottenere letture della batteria estremamente precise. Questo abiliterà il calcolo predittivo sui dati storici, evitando che i picchi di consumo del modem durante le trasmissioni falsino la percentuale di carica.
- **Design e protezione:** progettazione di un involucro su misura, ad esempio tramite modellazione 3D e stampaggio a iniezione. La scocca dovrà avere una certificazione minima **IP67** o **IP68** per garantire totale impermeabilità (ad acqua e fango) e resistere agli urti causati dai movimenti dell'animale.

8 Conclusioni

Riferimenti bibliografici

- [1] *T-SIM7670G-S3 Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/docs/en/esp32s3/sim7670g-s3/REAMDE.MD>.
- [2] *LTE Antenna Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/LTE%20Antenna%20Specifications.pdf>.
- [3] *GPS Antenna Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/GPS%20Antenna%20Specifications.pdf>.
- [4] *BMA400 CJMCU-400 Datasheet*. URL: https://www.bosch-sensortec.com/media/boschsensortec/downloads/product_flyer/bst-bma400-fl000.pdf.
- [5] *PocketBase Documentation*. URL: <https://pocketbase.io/docs/>.
- [6] *ngrok Documentation - HTTP Tunnels*. URL: <https://ngrok.com/docs/secure-tunnels/>.
- [7] *rclone - rsync for cloud storage*. URL: <https://rclone.org/>.
- [8] *Backblaze B2 Cloud Storage*. URL: <https://www.backblaze.com/b2/cloud-storage.html>.
- [9] *UptimeRobot - Free Website Monitoring Service*. URL: <https://uptimerobot.com/>.
- [10] *Flutter Documentation*. URL: <https://docs.flutter.dev/>.
- [11] *Nominatim Documentation*. URL: <https://nominatim.org/release-docs/latest/>.
- [12] *Node.js Documentation*. URL: <https://nodejs.org/en/docs/>.
- [13] *Firebase Admin SDK - Cloud Messaging*. URL: <https://firebase.google.com/docs/cloud-messaging/server>.